

AI Quiz Generator from PDF

OJT Coding Exam — Acceptance Criteria

Adaptive Hub Exam | Stack: Laravel + Laragon + MySQL + Groq AI | No Auth | Time Limit: 4 Hours

Overview

The OJT candidate must build a Laravel web application running locally via Laragon. The system allows any user to upload a PDF module or textbook chapter, which is then sent to the Groq API (free tier, model:

llama3-70b-8192) to generate 15 multiple choice questions with 4 choices each, a correct answer, and a brief explanation per question. Generated quizzes are saved to the database and can be viewed or deleted. No user authentication is required.

System Architecture

- Backend: Laravel (latest stable) running on Laragon (local environment)
- Database: MySQL via Laragon — migrations required
- No user authentication — the app is publicly accessible
- PDF parsing: server-side using smalot/pdfparser (Composer package)
- AI: Groq API — model llama3-70b-8192 (free tier) — called from a Laravel Service class
- Frontend: Blade templates — no separate frontend framework required
- GROQ_API_KEY stored in .env only — never in source code

1. PDF Upload

#	Acceptance Criteria	Priority	Evaluation Method
AC-01	A file upload form accepts PDF files only (.pdf extension enforced)	Must Have	Try uploading .docx or .jpg — should be rejected
AC-02	Uploaded PDF is stored server-side in storage/app/pdfs	Must Have	Check storage folder after upload
AC-03	PDF text is extracted server-side using smalot/pdfparser	Must Have	Code review + upload test with sample PDF
AC-04	If the PDF cannot be read, a clear error message is shown to the user	Must Have	Upload empty or corrupted PDF — check error message
AC-05	PDF file size is validated — reject files larger than 100MB	Should Have	Upload a file over 100MB — expect validation error

2. Groq AI Integration

#	Acceptance Criteria	Priority	Evaluation Method
AC-01	Groq API is called server-side using model llama3-70b-8192 (free tier)	Must Have	Code review — confirm model name in QuizService
AC-02	The prompt sends the extracted PDF text and instructs Groq to return 15 MCQ questions in JSON format	Must Have	Code review of prompt in QuizService
AC-03	Each question in the JSON has: question text, 4 choices (A–D), correct answer, and explanation	Must Have	Inspect generated quiz in DB — check all fields
AC-04	Groq response is parsed as JSON — malformed output shows a user-friendly error	Must Have	Simulate bad response — confirm graceful error shown
AC-05	GROQ_API_KEY is in .env only — not hardcoded anywhere in source code	Must Have	Inspect all source files — key must not appear
AC-01	Groq API logic is inside a dedicated QuizService class — not in the controller	Should Have	Code review — no Groq logic inside QuizController

3. Quiz Management — CRUD

#	Acceptance Criteria	Priority	Evaluation Method
AC-02	Generated quiz is saved to the database with a title (from filename) and linked to the logged-in user	Must Have	Check quizzes table after generation
AC-03	Each of the 15 questions is saved to the questions table linked to the quiz	Must Have	Check questions table — 15 rows per quiz
AC-04	Each question row stores: question text, choice A, choice B, choice C, choice D, correct answer, explanation	Must Have	Inspect questions table schema and a sample row
AC-05	User can view a list of all their generated quizzes on the dashboard	Must Have	Visit dashboard — quizzes listed
AC-01	User can view a single quiz and see all 15 questions with choices and answer key	Must Have	Click a quiz — all questions displayed with answers
AC-02	User can delete a quiz — all associated questions are removed	Must Have	Delete quiz — confirm removed from DB with cascade
AC-03	A user cannot view or delete another user's quiz	Must Have	Login as User B — try to access User A's quiz — expect 403

4. Version Control

#	Acceptance Criteria	Priority	Evaluation Method
AC-03	Project is initialized as a Git repository (git init or cloned)	Must Have	Check for .git folder in project root
AC-04	A .gitignore file exists and excludes: .env, /vendor, /node_modules, /storage/app/pdfs	Must Have	Inspect .gitignore — confirm sensitive files excluded
AC-05	Candidate makes at least 5 meaningful commits throughout development — not just one final commit	Must Have	Run git log — check commit count and messages
AC-01	Commit messages are descriptive and follow a consistent format (e.g. feat: add PDF upload, fix: handle empty PDF error)	Should Have	Run git log --oneline — review message quality
AC-02	A .env.example file is committed to the repo with all required keys but no real values	Must Have	Check repo — .env.example present, .env absent

7. Database & Migrations

#	Acceptance Criteria	Priority	Evaluation Method
AC-03	Migrations exist for: quizzes and questions tables	Must Have	Run php artisan migrate — no errors
AC-04	quizzes table has: id, user_id (FK), title, pdf_path, created_at, updated_at	Must Have	Inspect quizzes migration file
AC-05	questions table has: id, quiz_id (FK cascade delete), question, choice_a, choice_b, choice_c, choice_d, correct_answer, explanation	Must Have	Inspect questions migration file
AC-01	Eloquent relationships defined: User hasMany Quizzes, Quiz hasMany Questions	Must Have	Code review of Quiz and User models

8. UI & Usability

#	Acceptance Criteria	Priority	Evaluation Method
AC-02	Dashboard shows a list of generated quizzes with title and date created	Must Have	Visual inspection
AC-03	Quiz detail page displays all 15 questions clearly numbered with choices and correct answer highlighted	Must Have	Visual inspection
AC-04	A loading message or indicator is shown while the PDF is being processed and AI is generating	Must Have	Observe during generation
AC-05	UI is clean and navigable — an evaluator can use it without instructions	Should Have	Evaluator usability test

Scoring Rubric

Category	Weight	Pass Threshold
PDF Upload (AC-01 to AC-05)	20%	AC-01, AC-02, AC-03, AC-04 passed
Groq AI Integration (AC-06 to AC-11)	25%	AC-06, AC-07, AC-08, AC-10 passed
Quiz CRUD (AC-12 to AC-18)	25%	AC-12, AC-13, AC-15, AC-16, AC-17 passed
Version Control (AC-23 to AC-27)	15%	AC-23, AC-24, AC-25, AC-27 passed
Database & Migrations (AC-28 to AC-31)	10%	AC-28, AC-29, AC-30, AC-31 passed
UI & Usability (AC-32 to AC-35)	5%	AC-32, AC-33, AC-34 passed
TOTAL	100%	70% or higher to pass

Overall passing score: 70% or higher. Must Have criteria in AC-01, AC-03, AC-06, AC-08, AC-10, AC-12, AC-13, AC-23, AC-25, and AC-28 must each be individually met regardless of total score.

Estimated Completion Time (4-Hour Target)

Module	Task Summary	Est. Time
Project Setup	Laragon, Laravel install, .env, DB config, Breeze auth scaffold	25 min
Git Setup	git init, .gitignore, initial commit, .env.example	10 min
Authentication	Test Breeze register/login/logout, protect routes	15 min
PDF Upload	File upload form, validation, storage, smalot/pdfparser	45 min
Groq Integration	QuizService, prompt, API call, JSON parsing, error handling	60 min
Database & Models	Migrations for quizzes + questions, Eloquent models + relations	30 min
Quiz CRUD	Save questions to DB, list view, detail view, delete cascade	40 min
UI & Blade Views	Dashboard, upload form, quiz detail page, loading state	25 min
Testing & Cleanup	End-to-end test, meaningful commits, README setup steps	10 min
TOTAL		4 hrs

This scope is achievable in 4 hours for an OJT candidate who has basic Laravel familiarity. The Groq AI integration is the most time-sensitive step — the candidate should prioritize getting it working early (step 3) before polishing the UI.

Exam Notes & Deliverables

- Time limit: 4 hours
- Environment: Laravel running on Laragon (local) — no deployment required
- AI Provider: Groq API only — model must be llama3-70b-8192 (free tier)
- Groq free API key: register at console.groq.com — candidate must create their own account
- PDF input: upload as a file — plain text paste is not accepted
- Frontend: Blade templates only — no React, Vue, or separate SPA required
- Version control: Git is required — project must be a Git repository with meaningful commit history
- Minimum 5 commits required — evaluator will run git log to verify
- .env must NOT be committed — only .env.example with blank values
- Candidate must submit: full source code, .env.example, and a short README with setup steps
- The evaluator will run `php artisan migrate` and `php artisan serve` to test the submission
- Reference implementation must not be shared with the candidate before the exam

Prepared by Adaptive Hub Team